

The Weighted All-Pairs-Shortest-Path-Length Problem on Two-Terminal Series-Parallel Graphs

William Chung-Kung Yen
Department of Information Management
Shih Hsin University, Taipei, Taiwan
(e-mail: ckyen001@ms7.hinet.net)

Shuo-Cheng Hu
Department of Information Management
Shih Hsin University, Taipei, Taiwan
(e-mail: schu@cc.shu.edu.tw)

Tai-Chang Chen
National Taiwan University of Science and Technology
Taipei, Taiwan
(e-mail: bjchente@gmail.com)

Abstract

Let $G(V, E)$ be a n -vertex graph with $V = \{v_1, \dots, v_n\}$ and each edge $e = (v_i, v_j)$ is associated with two real weights, $W(v_i \rightarrow v_j)$ and $W(v_j \rightarrow v_i)$. Let P be a path from any source vertex s to any destination vertex d and assume that P is $s = u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_t = d, t \geq 2$. The length of P , denoted by $\text{Len}(P)$, is defined as $\sum_{j=1}^{t-1} W(u_j \rightarrow u_{j+1})$.

Given any two distinct vertices s and d , the length from s to d is defined as $\text{Len}(s \Rightarrow d) = \min \{ \text{Len}(P) \mid P \text{ is a path from } s \text{ to } d \}$. $\text{Len}(d \Rightarrow s)$ can be defined similarly. We define $\text{Len}(s \Rightarrow s) = 0$, for all $s \in V$. In [8], Dr. M. R. Henzinger et. al. shown that $\text{Len}(s \Rightarrow d)$ and $\text{Len}(d \Rightarrow s)$, for all vertices s and d , can be computed in $O(n^2)$ time on planar graphs with nonnegative edge-weights. But the time-complexity soon increases to $O(n^{7/3} \log(nL))$ if negative edge-weights are allowed, where L is the absolute value of the most negative edge-weight. When negative edge-weights are allowed, this paper shows a meaningful improvement: the time-complexity remain to $O(n^2)$ on two-terminal series-parallel graphs, which form an important subclass of planar graphs.

Keywords: weighted all-pairs-shortest-path-length problem, two-terminal series-parallel graphs, time-optimal algorithm

1. Introduction

Computing shortest paths between all pairs of vertices of a connected directed graph with weights on edges is a very essential and fundamental graph problem. Traditionally, the edge-weights are nonnegative and the problem is called the *All-Pairs Shortest Paths problem* (the *APSP problem*). For general graphs with n vertices and m edges, some well-know early algorithms, such as Dijkstra's algorithm and Floyd's algorithm, solved the APSP problem in $O(n^3)$ -time [3]. After then, some improved algorithms were proposed, such as an $O(mn + n^2 \log n)$ -time algorithm in [5], an $O(mn + n^2 \log \log n)$ -time algorithm in [14], an $O(n^3 \log \log n / \log n)$ -time algorithm in [17], an $O(n^3 (\log \log n / \log n)^{5/7})$ -time algorithm in [7]. Furthermore, the APSP problem on interval graphs with unit-distance edge-weights can be solved in $O(n^2)$ -time [13, 15]. In [6], the author proposed an $O(m + n \log n / \log \log n)$ implementation of Dijkstra's algorithm. In [18], the issue on Single-Source Shortest Paths with positive integer weights was studied. Recently, the authors in [1] proposed a high performance Divide-and-Conquer algorithm and illustrated useful experimental results and the authors in [2] studied the approximation schemes for the problem.

Instead of identifying all shortest paths, this paper studies the very related problem, called the *All-Pairs*

Shortest Paths Lengths problem (the *APSPL problem*), which is just to compute the lengths of the shortest paths between all pairs of vertices of the input graph. In [9], the authors showed that the APSPL problem can be solved in $O(n^2)$ -time on unweighted chordal bipartite graphs. In this paper, $G(V, E, W)$ denotes a n -vertex graph with $V = \{v_1, \dots, v_n\}$ and each edge $e = (v_i, v_j)$ has two real weights, $W(v_i \rightarrow v_j)$ and $W(v_j \rightarrow v_i)$. Let P be a path from any source vertex s to any destination vertex d and assume that P is $s = u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_t = d$, $t \geq 2$. The length of P , denoted by $\text{Len}(P)$, is

defined as $\sum_{j=1}^{t-1} W(u_j \rightarrow u_{j+1})$. Given any two

distinct vertices s and d , the length from s to d is defined as $\text{Len}(s \Rightarrow d) = \min\{\text{Len}(P) \mid P \text{ is a path from } s \text{ to } d\}$ and $\text{Len}(d \Rightarrow s)$ can be defined symmetrically. We define $\text{Len}(s \Rightarrow s) = 0$, for all $s \in V$. The problem studied in this paper is as follows:

The Weighted All-Pairs Shortest Paths Lengths Problem (The WAPSPL Problem): Given a graph $G(V, E, W)$ with $V = \{v_1, \dots, v_n\}$, compute $\text{Len}(v_i \Rightarrow v_j)$ and $\text{Len}(v_j \Rightarrow v_i)$, for all pair of vertices v_i and v_j .

The rest of the paper is organized as follows. Section 2 will describe the notations and background of TTSP graphs. Then, an $O(n^2)$ -time algorithm for the WAPSPL problem will be designed in Section 3. Finally, the conclusions and future research issues will be discussed in Section 4.

2. Notations and Background

The class of *Two-Terminal Series-Parallel graphs* (TTSP graphs) can be defined recursively as follows [4, 10, 11, 16]: (1) A graph G consisting of a single edge $e = (v_i, v_j)$ is a TTSP graph. v_i and v_j are called the *left terminal* and the *right terminal* of G , respectively. In the rest of this paper, we use $\text{lt}(G)$ and $\text{rt}(G)$ to denote the two terminals of G , respectively, i.e., $\text{lt}(G) = v_i$ and $\text{rt}(G) = v_j$. (2) If H and Q are two TTSP graphs, then the new graph G derived by one of the following rules is also a TTSP graph. (a) G is derived by the *series-connection* of H and Q via identifying $\text{rt}(H)$ with $\text{lt}(Q)$. The new terminals of G are $\text{lt}(G) = \text{lt}(H)$ and $\text{rt}(G) = \text{rt}(Q)$. We denote $G = H \bullet Q$. (b) G is derived by the

parallel-connection of H and Q via identifying $\text{lt}(H)$ with $\text{lt}(Q)$ and $\text{rt}(H)$ with $\text{rt}(Q)$, respectively. The new terminals of G are $\text{lt}(G) = \text{lt}(H) = \text{lt}(Q)$ and $\text{rt}(G) = \text{rt}(H) = \text{rt}(Q)$. We denote $G = H + Q$.

Fig. 1 shows two TTSP graphs H and Q . Fig. 2 and Fig. 3 depict the TTSP graphs $G_1 = H \bullet Q$ and $G_2 = H + Q$, respectively.

A linear-time algorithm exists to recognize whether a graph G is a TTSP graph. A *paring tree* T is also generated to illustrate how the graph G is constructed via series and parallel connections [11]. The leaves of T correspond to all edges of G . Since a general tree can be interpreted as a binary tree easily [12], this paper only considers binary parsing trees.

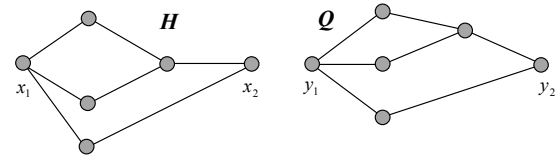


Figure 1: TTSP graphs H and Q with $\text{lt}(H) = x_1$, $\text{rt}(H) = x_2$ and $\text{lt}(Q) = y_1$, $\text{rt}(Q) = y_2$

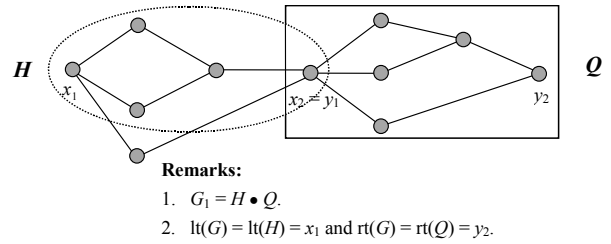


Figure 2: The TTSP graph G_1 derived by the series-connection of H and Q .

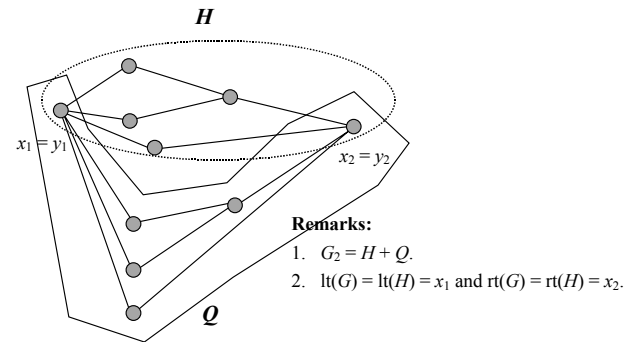


Figure 3: The TTSP graph G_2 derived by the parallel-connection of H and Q .

Given a TTSP graph G , let T be its parsing tree and r is the root of T . For any node z of T , the subtree rooted at z is denoted by $T(z)$. It is easy to verify that the subgraph induced by the leaves of $T(z)$ is also a TTSP graph. So, we use $G(z)$ to denote the subgraph induced by the leaves of $T(z)$, and $V(G(z))$ and $E(G(z))$ denote its vertex-set and the edge-set,

respectively. By this notation, $G(r)$ is just the original input graph G . In another, each non-leaf node u of T is associated with a label $\text{lb}(u)$. Assume that q and f are the children of u . If $G(u) = G(q) \bullet G(f)$, then $\text{lb}(u) = S$ and u is called a *S-node*. Otherwise, it implies that $G(u) = G(q) + G(f)$. In this case, $\text{lb}(u) = P$ and u is called a *P-node*. Fig. 4 shows a TTSP graph G and a corresponding parsing tree T .

In [19], the authors investigated literatures on TTSP graphs for some problems, such as the minimum feedback vertex problem and the maximum independent set. Meanwhile, in [8], the authors shown that APSPL problem on planar graphs with nonnegative edge-weights can be solved in $O(n^2)$ time. But the time-complexity soon increases to $O(n^{\frac{7}{3}} \log(nL))$ if negative edge-weights are allowed, where L is the absolute value of the most negative edge-weight. The class of TTSP graphs is an important subclass of planar graphs. This paper will establish a meaningful improvement: the WAPSPL problem on TTSP graphs can be solved in $O(n^2)$ time.

3. An $O(n^2)$ -Time Algorithm

It is so trivial to solve our problem when the input graph consists only a single edge. Therefore, we assume that the input graph G consists of at least two edges hereafter. To go on our discussion, the following definitions are introduced firstly.

Definition 1: For any node z of $T(r)$, the nodes in the path from z to the root r , except z itself, are called the *ancestors* of z .

Definition 2: For any two distinct vertices v_i and v_j of $G(r)$, the *lowest common ancestor* of v_i and v_j , denoted as $\text{LCA}(\{v_i, v_j\})$ is defined as follows: (1) If (v_i, v_j) is an edge of $G(r)$, then it implies that (v_i, v_j) corresponds to some leaf node z and $\text{LCA}(\{v_i, v_j\})$ is defined to be z itself. (2) If (v_i, v_j) is not an edge of $G(r)$, then there must exist two leaf nodes p and q such that v_i is the one vertex of the edge corresponding to p and v_j is the one vertex of the edge corresponding to q . In this situation, $\text{LCA}(\{v_i, v_j\})$ is defined to be the node u farthest from r such that u is a ancestor of both p and q .

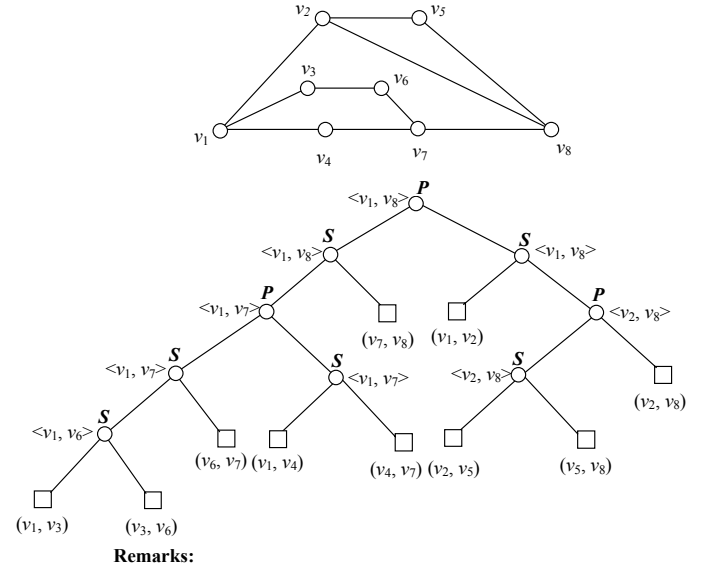


Figure 4: A TTSP graph G and its corresponding parsing tree T .

Definition 3: Suppose that u is a *P-node* of $T(r)$. u is called a *bottom P-node* if all nodes of $T(u)$ are either *S-nodes* or leaf nodes, except u itself. Otherwise, u is called a *non-bottom P-node*. In addition, u is called a *top P-node* if the ancestors of u in $T(r)$ are all *S-nodes*. If u is not a top *P-node*, then u is called a *non-top P-node*. By this definition, u can be a top *P-node* and a bottom *P-node* at the same time.

Definition 4: For any bottom *P-node* u of $T(r)$, $\text{pair_set}(u)$ is defined as $\{\{v_i, v_j\} \mid v_i, v_j \in V(G(u))\}$

Definition 5: For any non-bottom *P-node* u of $T(r)$, $\text{pair_set}(u)$ is defined as $\{\{v_i, v_j\} \mid v_i, v_j \in V(G(u))\} - \bigcup_{\text{for all } P\text{-node } v \neq u \text{ of } T(u)} \text{pair_set}(v)$.

Definition 6: For each non-top *P-node* u , the *lowest P-node ancestor* of u , denoted as $\text{LPNA}(u)$ is the *P-node* f satisfying the following conditions: (1) f is nearest to u , and (2) f is in the path from u to the root r .

Definition 7: For each non-leaf node z of $T(r)$, let q and f be the left child and the right child of z , respectively. Define $\text{left_descendants}(z) = \{x \mid x \text{ is a node in } T(q)\}$ and $\text{right_descendants}(z) = \{x \mid x \text{ is a node in } T(f)\}$.

The following lemma can be easily established.

Lemma 1: For any two distinct vertices v_i, v_j of $G(r)$, there exists at most one *P-node* u such that

$\{v_i, v_j\} \in \text{pair_set}(u)$. If no P -node u such that $\text{pair_set}(u)$ contains $\{v_i, v_j\}$, then $\text{LCA}(\{v_i, v_j\})$ must be either a leaf node or a S -node, and all ancestors of $\text{LCA}(\{v_i, v_j\})$ must also be S -nodes. Otherwise, the only P -node containing $\{v_i, v_j\}$ will be denoted as $\text{PN}(\{v_i, v_j\})$. Meanwhile, $\text{pair_set}(u)$ for all P -nodes u can be computed in $O(n^2)$ time.

Since our objective is to compute $\text{Len}(v_i \Rightarrow v_j)$ and $\text{Len}(v_j \Rightarrow v_i)$, for all distinct vertices v_i and v_j of $G(r)$, The following reasoning can then be made based on Lemma 1.

Definition 8: For any two distinct vertices v_i, v_j of $G(r)$, if no P -node u such that $\text{pair_set}(u)$ contains $\{v_i, v_j\}$, then the pair $\{v_i, v_j\}$ is called a *Type I pair*. Otherwise, there exists only one P -node u such that $\{v_i, v_j\} \in \text{pair_set}(u)$ and the pair $\{v_i, v_j\}$ is called a *Type II pair*.

Lemma 2: For any two distinct vertices v_i, v_j of $G(r)$, $\{v_i, v_j\}$ is either a Type I pair or a Type II pair.

Definition 9: For any node z of $T(r)$ and any two distinct vertices v_i and v_j of $G(z)$, define $\text{Len}^{(z)}(v_i \Rightarrow v_j) = \min\{\text{Len}(P) \mid P \text{ is a path from } v_i \text{ to } v_j \text{ consisting of the edges only in } E(G(z)).\}$ and $\text{Len}^{(z)}(v_j \Rightarrow v_i) = \min\{\text{Len}(P) \mid P \text{ is a path from } v_j \text{ to } v_i \text{ consisting of the edges only in } E(G(z)).\}$

Definition 10: For any P -node u of $T(r)$ and any two distinct vertices v_i and v_j of $G(u)$, define $\text{OutLen}(v_i \Rightarrow v_j) = \min\{\text{Len}(P) \mid P \text{ is a path from } v_i \text{ to } v_j \text{ consisting of the edges only in } E - E(G(u)).\}$ and $\text{OutLen}(v_j \Rightarrow v_i) = \min\{\text{Len}(P) \mid P \text{ is a path from } v_j \text{ to } v_i \text{ consisting of the edges only in } E - E(G(u)).\}$ If there exists no path from v_i to v_j consisting of the edges only in $E - E(G(u))$, then

set $\text{OutLen}(v_i \Rightarrow v_j) = \infty$. Also, if there exists no path from v_j to v_i consisting of the edges only in $E - E(G(u))$, then set $\text{OutLen}(v_j \Rightarrow v_i) = \infty$.

Lemma 3: For any type I pair $\{v_i, v_j\}$ of $G(r)$, let $u = \text{LCA}(\{v_i, v_j\})$. We must have $\text{Len}(v_i \Rightarrow v_j) = \text{Len}^{(u)}(v_i \Rightarrow v_j)$ and $\text{Len}(v_j \Rightarrow v_i) = \text{Len}^{(u)}(v_j \Rightarrow v_i)$.

Proof: Let $u \xrightarrow{[1]} \dots \xrightarrow{[2]} \dots \xrightarrow{[\alpha]} r$ be the path from u to r in $T(r)$. It implies that $u^{[1]}, u^{[2]}, \dots, u^{[\alpha]}$ are the ancestors of u and they must be all S -nodes. By the definitions of TTSP graphs and their parsing trees, $G(r)$ must be constructed via series-connections marked at $\text{lb}(u^{[1]})$, $\text{lb}(u^{[2]})$, $\dots$, $\text{lb}(u^{[\alpha]})$, respectively. It is easy to verify that $\text{Len}(v_i \Rightarrow v_j) = \text{Len}^{(u)}(v_i \Rightarrow v_j)$ and $\text{Len}(v_j \Rightarrow v_i) = \text{Len}^{(u)}(v_j \Rightarrow v_i)$. ■

Lemma 4: For any type II pair $\{v_i, v_j\}$ of $G(r)$, let $u = \text{PN}(\{v_i, v_j\})$. In addition, let $z = \text{LPNA}(u)$ and $v_s = \text{lt}(G(u))$ and $v_t = \text{rt}(G(u))$. Then, we have the following formulas.

$$\begin{aligned} \text{Len}(v_i \Rightarrow v_j) &= \min\{\text{Len}^{(u)}(v_i \Rightarrow v_j), \\ &\quad \text{Len}^{(a)}(v_i \Rightarrow v_s) + \text{OutLen}(v_s \Rightarrow v_t) \\ &\quad + \text{Len}^{(b)}(v_t \Rightarrow v_j), \\ &\quad \text{Len}^{(a)}(v_i \Rightarrow v_t) + \text{OutLen}(v_t \Rightarrow v_s) \\ &\quad + \text{Len}^{(b)}(v_s \Rightarrow v_j)\} \end{aligned}$$

$$\begin{aligned} \text{Len}(v_j \Rightarrow v_i) &= \min\{\text{Len}^{(u)}(v_j \Rightarrow v_i), \\ &\quad \text{Len}^{(b)}(v_j \Rightarrow v_s) + \text{OutLen}(v_s \Rightarrow v_t) \\ &\quad + \text{Len}^{(a)}(v_t \Rightarrow v_i), \\ &\quad \text{Len}^{(b)}(v_j \Rightarrow v_t) + \text{OutLen}(v_t \Rightarrow v_s) \\ &\quad + \text{Len}^{(a)}(v_s \Rightarrow v_i)\} \end{aligned}$$

Proof: Since $\text{Len}(v_i \Rightarrow v_j)$ and $\text{Len}(v_j \Rightarrow v_i)$ are just symmetrical by interchanging the roles of v_i and v_j . We just show the correctness of the

formula about $\text{Len}(v_i \Rightarrow v_j)$. The following cases are considered, respectively.

Case 1. u is a top P -node

It is easy to ascertain $\text{OutLen}(v_s \Rightarrow v_t) = \text{OutLen}(v_t \Rightarrow v_s) = \infty$, i.e., each path P from v_i to v_j consists of only edges in $E(G(u))$. Thus, the formula holds for this case.

Case 2. u is a non-top P -node

In this case, at least one ancestor of u is also a P -node. By the construction rules of TTSP graphs, at least one path from v_s to v_t consists of the edges only in $E - E(G(u))$ and at least one path from v_t to v_s consists of the edges only in $E - E(G(u))$. Therefore, it is easy to verify that the formula holds. ■

Based upon all reasoning so far, the following algorithm can solve the WAPSPL on TTSP graphs correctly.

Algorithm WAPSPL_TTSP

Input: A weighted TTSP graph G and its corresponding parsing tree $T(r)$ such that $\text{pair_set}(u)$, $\text{left_descendants}(u)$, $\text{right_descendants}(u)$, and $\text{LPAN}(u)$, for all P -nodes u , have been computed.
/* We assume that the vertices of $G(r)$ are $v_1, \dots, v_n$. */

Output: The array $L[1 \dots n, 1 \dots n]$ such that $L[i, j] = \text{Len}(v_i \Rightarrow v_j)$ and $L[j, i] = \text{Len}(v_j \Rightarrow v_i)$, for all $1 \leq i, j \leq n$.
/* The array L is global to all phases of the algorithm. */

Phase I:

$L[i, j] = L_1[i, j] = L_2[i, j] = 0, 1 \leq i \leq n, 1 \leq j \leq n$;

for each P -node u of $T(r)$

 Compute $L_1(\text{rt}(G(u)) \Rightarrow \text{lt}(G(u)))$,
 $L_2(\text{rt}(G(u)) \Rightarrow \text{lt}(G(u)))$, $L_1(\text{lt}(G(u)) \Rightarrow \text{rt}(G(u)))$, and $L_2(\text{lt}(G(u)) \Rightarrow \text{rt}(G(u)))$;

$\text{Len}^{(u)}(\text{rt}(G(u)) \Rightarrow \text{lt}(G(u))) = \min\{L_1(\text{rt}(G(u)) \Rightarrow \text{lt}(G(u))), L_2(\text{rt}(G(u)) \Rightarrow \text{lt}(G(u)))\}$;

$\text{Len}^{(u)}(\text{lt}(G(u)) \Rightarrow \text{rt}(G(u))) = \min\{L_1(\text{lt}(G(u)) \Rightarrow \text{rt}(G(u))), L_2(\text{lt}(G(u)) \Rightarrow \text{rt}(G(u)))\}$;

endfor

for each S -node z of $T(r)$

 Compute $\text{Len}^{(z)}(v_i \Rightarrow v_j)$ and $\text{Len}^{(z)}(v_j \Rightarrow v_i)$, for all v_i and v_j of $G(z)$;

endfor

/* For simplicity, we use two two-dimensional arrays $L_1[1 \dots n, 1 \dots n]$ and $L_2[1 \dots n, 1 \dots n]$ to store $L_1(v_s \Rightarrow v_t)$, $L_2(v_s \Rightarrow v_t)$, $L_1(v_t \Rightarrow v_s)$, and $L_2(v_t \Rightarrow v_s)$, for all v_s and v_t such that $\langle v_s, v_t \rangle$ are terminals of some P -node u . Meanwhile, these two arrays are global to all phases of the algorithm. */

Phase II:

for each P -node u of $T(r)$

 Compute $\text{OutLen}(\text{rt}(G(u)) \Rightarrow \text{lt}(G(u)))$ and $\text{OutLen}(\text{lt}(G(u)) \Rightarrow \text{rt}(G(u)))$;

endfor

/* If there is no P -node in $T(r)$, then this phase does nothing. */

/* For simplicity, we also use a two-dimensional array $OL[1 \dots n, 1 \dots n]$ to store $\text{OutLen}(v_s \Rightarrow v_t)$ and $\text{OutLen}(v_t \Rightarrow v_s)$, for all v_s and v_t such that $\langle v_s, v_t \rangle$ are terminals of some P -node u . Meanwhile, $OL[i, j]$ is initialized to be ∞ , for all $1 \leq i, j \leq n$. and this array is global to all phases of the algorithm. */

Phase III:

for each P -node u of $T(r)$

for each pair $\{v_i, v_j\}$ in $\text{pair_set}(u)$

 Use the arrays OL , L_1 , and L_2 to adjust

$L[i, j]$ for obtaining $\text{Len}(v_i \Rightarrow v_j)$;

endfor

endfor

End WAPSPL_TTSP

3.1 Phase I

For each node z of $T(r)$, let $\text{lt}(G(z)) = v_i$ and $\text{rt}(G(z)) = v_j$. The following cases should be considered.

Case 1. z is a leaf node

In this case, we just let $L[i, j] = W(v_i \rightarrow v_j)$ and $L[j, i] = W(v_j \rightarrow v_i)$.

Case 2. z is a S -Node

Let q and f be the left child and the right child of z , respectively, i.e., $G(z) = G(q) \bullet G(f)$. In addition, let

$v_i = \text{lt}(G(q))$ and $v_k = \text{rt}(G(q))$; and let $v_k = \text{lt}(G(f))$ and $v_j = \text{rt}(G(f))$. The following codes will be performed.

```

recursively compute  $L[s, t]$ , and  $L[t, s]$ , for  $v_s$ ,
 $v_t \in V(G(q))$ ;
recursively compute  $L[s, t]$ , and  $L[t, s]$ , for  $v_s$ ,
 $v_t \in V(G(f))$ ;
 $L[i, j] = L[i, k] + L[k, j]$ ;
 $L[j, i] = L[j, k] + L[k, i]$ ;
for each pair of vertices  $v_a \neq v_b$  such that  $v_a \in$ 
 $V(G(q)) - \{v_i, v_k\}$  and  $v_b \in V(G(f)) - \{v_k,$ 
 $v_j\}$ 
/* If one of  $V(G(q)) - \{v_i, v_k\}$  and  $V(G(f)) -$ 
 $\{v_k, v_j\}$  is empty, then we just skip this loop. */
 $L[a, b] = L[a, k] + L[k, b]$ ;
 $L[b, a] = L[b, k] + L[k, a]$ ;
endfor
for each pair of vertices  $v_i$  and  $v_b$  such that  $v_b$ 
 $\in V(G(f)) - \{v_k, v_j\}$ 
/* If  $V(G(f)) - \{v_k, v_j\}$  is empty, then we just
skip this loop. */
 $L[i, b] = L[i, k] + L[k, b]$ ;
 $L[b, i] = L[b, k] + L[k, i]$ ;
endfor
for each pair of vertices  $v_a$  and  $v_j$  such that  $v_a$ 
 $\in V(G(q)) - \{v_i, v_k\}$ 
/* If  $V(G(q)) - \{v_i, v_k\}$  is empty, then we just
skip this loop. */
 $L[a, j] = L[a, k] + L[k, j]$ ;
 $L[j, a] = L[j, k] + L[k, a]$ ;
endfor

```

Case 3. z is a P -node

Let q and f be the left child and the right child of z , respectively, i.e., $G(z) = G(q) + G(f)$. In this case, $\text{lt}(G(q)) = \text{lt}(G(f)) = v_i$ and $\text{rt}(G(q)) = \text{rt}(G(f)) = v_j$.

The following codes will be performed.

```

recursively compute  $L[s, t]$ , and  $L[t, s]$ , for  $v_s$ ,
 $v_t \in V(G(q))$ ;
 $L_1[i, j] = L[i, j]$ ;  $L_1[j, i] = L[j, i]$ ;
recursively compute  $L[s, t]$ , and  $L[t, s]$ , for  $v_s$ ,
 $v_t \in V(G(f))$ ;
 $L_2[i, j] = L[i, j]$ ;  $L_2[j, i] = L[j, i]$ ;

```

$$L[i, j] = \min\{L_1[i, j], L_2[i, j]\};$$

$$L[j, i] = \min\{L_1[j, i], L_2[j, i]\};$$

Lemma 5: Phase I can be done in $O(n^2)$ time.

Proof: To complete Phase I, each pair of vertices v_i and v_j is examined constant times and each node of $T(r)$ is also examined constant times. This implies that Phase I can be done in $O(n^2)$ time. ■

3.2 Phase II

It is easy to see that $\text{LPNA}(u)$, for each P -node u , can be obtained by the post-order traversal on T and the time-complexity is linear. Based on the reasoning and the results of so far, Phase II can be achieved by breadth-first traversal on T from the root r . For each non-top P -node u , assume that $f = \text{LPNA}(u)$ and the terminals of $G(u)$ and $G(f)$ are $\langle v_i, v_j \rangle$ and $\langle v_s, v_t \rangle$, respectively. The following code will be performed.

```

if ( $u \in \text{left\_descendants}(f)$ )
 $OL[i, j] = L[i, s] + L_2[s, t] + L[t, j]$ ;
 $OL[j, i] = L[j, t] + L_2[t, s] + L[s, i]$ ;
else /*  $u \in \text{right\_descendants}(f)$ . */
 $OL[i, j] = L[i, s] + L_1[s, t] + L[t, j]$ ;
 $OL[j, i] = L[j, t] + L_1[t, s] + L[s, i]$ ;
endif

```

The following lemma can be verified easily.

Lemma 6: Phase II can be done in $O(n^2)$ -time.

3.3 Phase III

This phase will be achieved by depth-first traversal on T from the root r . For each P -node u , let q and f be the left child and the right child of u , respectively, i.e., $G(u) = G(q) + G(f)$. In this case, $\text{lt}(G(q)) = \text{lt}(G(f)) = v_i$ and $\text{rt}(G(q)) = \text{rt}(G(f)) = v_j$.

The following codes will be performed.

```

for each pair  $\{v_a, v_b\} \in \text{pair\_set}(u)$ 
docase
case  $v_a, v_b \in V(G(q))$ 
 $L[a, b] = \min\{L[a, b],$ 
 $L[a, i] + L_2[i, j] + L[j, b],$ 
 $L[a, j] + L_2[j, i] + L[i, b],$ 
 $L[a, i] + OL[i, j] + L[j, b],$ 
 $L[a, j] + OL[j, i] + L[i, b]\}$ ;
 $L[b, a] = \min\{L[a, b], L[b, i] + L_2[i, j]$ 

```

```

      + L[j, a], L[b, j] + L2[j, i] + L[i, a],
      L[b, i] + OL[i, j] + L[j, a], L[b, j] +
      OL[j, i] + L[i, a]};
break;
case  $v_a, v_b \in V(G(f))$ 
   $L[a, b] = \min\{L[a, b],$ 
     $L[a, i] + L_1[i, j] + L[j, b],$ 
     $L[a, j] + L_1[j, i] + L[i, b],$ 
     $L[a, i] + OL[i, j] + L[j, b],$ 
     $L[a, j] + OL[j, i] + L[i, b]\};$ 
   $L[b, a] = \min\{L[a, b], L[b, i] + L_1[i, j] + L[j,$ 
     $a], L[b, j] + L_1[j, i] + L[i, a], L[b, i] +$ 
     $OL[i, j] + L[j, a], L[b, j] + OL[j, i] +$ 
     $L[i, a]\};$ 
break;
case ( $v_a \in V(G(q))$  and  $v_b \in V(G(f))$ ) or ( $v_a \in$ 
   $V(G(f))$  and  $v_b \in V(G(q))$ )
   $L[a, b] = \min\{L[a, i] + L[i, b],$ 
     $L[a, j] + L[j, b],$ 
     $L[a, i] + OL[i, j] + L[j, b],$ 
     $L[a, j] + OL[j, i] + L[i, b]\};$ 
   $L[b, a] = \min\{L[b, i] + L[i, a],$ 
     $L[b, j] + L[j, a],$ 
     $L[b, i] + OL[i, j] + L[j, a],$ 
     $L[b, j] + OL[j, i] + L[i, a]\};$ 
break;
endcase
endfor

```

Lemma 7: Phase III can be done in $O(n^2)$ -time.

All reasoning and results so far can easily imply the following theorem.

Theorem 1: The weighted APSPL problem on TTSP graphs can be solved in $O(n^2)$ time.

4. Conclusions

In [8], the authors shown that APSPL problem on planar graphs with nonnegative edge-weights can be solved in $O(n^2)$ time. But the time-complexity soon increases to $O(n^{7/3} \log(nL))$ if negative edge-weights are allowed, where L is the absolute value of the most negative edge-weight. The class of TTSP graphs is a vital subclass of the class of planar graphs. This paper has established a meaningful improvement: the WAPSPL problem on TTSP graphs can be solved in $O(n^2)$ time. Our algorithm

is time-optimal in worst case since there are $\binom{n}{2} =$

$O(n^2)$ pairs of distinct vertices in any graph G .

In the future, it is very important and worthy to extend our result to the classes of graphs with the property $m = O(n)$, such as planar graphs. In another, studying the WAPSPL on other classes of graphs such as bipartite graphs, permutation graphs, is also a practical and interesting issue.

References

- [1] P. D' Alberto and A. Nicolau, "R-Kleene: A High-Performance Divide-and-Conquer Algorithm for the All-Pair Shortest Path for Densely Connected Networks", *Algorithmica*, Vol. 47, pp. 203-213, 2007.
- [2] E. Cohen, "Polylog-Time and Near-Linear Work Approximation Scheme for Undirected Shortest Paths", *Journal of the ACM*, Vol. 47, No. 1, January 2000.
- [3] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs", *Numerische Mathematik*, Vol. 1, pp. 269-271, 1959.
- [4] R. J. Duffin, "Topology of Series-Parallel Networks", *J. Math., Appl.* 10, pp. 303-318, 1965.
- [5] M. L. Fredman and R. E. Tarjan, "Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms", *Journal of ACM*, Vol. 34, pp. 596-615, 1987.
- [6] M. L. Fredman and D. E. Willard, "Trans-dichotomous Algorithms for Minimum Spanning Trees and Shortest Paths", *Journal of Computer and System Sciences*, Vol. 48, pp. 533-551, 1994.
- [7] Y. Han, "Improved Algorithm for All Pairs Shortest Paths", *Information Processing Letters*, Vol. 91, pp. 245-250, 2004.
- [8] M. R. Henzinger, P. Klein, S. Rao, and S. Subramanian, "Faster Shortest-Path Algorithms for Planar Graphs", *Journal of Computer and System Sciences*, Vol. 55, Iss. 1, pp. 3-23, 1997.
- [9] C-W Ho and J-M Chang, "Solving The All-pairs-shortest-length Problem on Chordal Bipartite Graphs", *Information Processing Letters*, Vol. 69, pp. 87-93, 1999.
- [10] R. H. Jan, L. H. Hsu, and Y. Y. Lee, "The Most Vital Edges with Respect to the Number of Spanning Trees in Two-Terminal Series-Parallel Graphs", *Bit*, Vol. 32, pp. 403-412, 1992.
- [11] T. Kikuno, N. Yoshida, and Y. Kakuda, "A Linear Algorithm for the Domination Number

- of a Series-Parallel Graph”, *Discrete Applied Mathematics*, Vol. 5, pp. 299-311, 1983.
- [12] D. E. Knuth, *The Art of Computer Programming, Vol. 1, Fundamental Algorithms*, Addison-Wesley, 1968.
 - [13] P. Mirchandani, “A Simple $O(n^2)$ Algorithm for the All-Pairs Shortest Path Problem on an Interval Graph”, *Networks*, Vol. 27, pp. 215-217, 1996.
 - [14] S. Pettie, “A Faster All-Pairs Shortest Path Algorithm for Real-Weighted Sparse Graphs”, *Lecture Notes in Computer Science*, Vol. 2380, pp. 85-97, 2002.
 - [15] R. Ravi, M. V. Marathe, and C. P. Rangan, “An Optimal Algorithm to Solve the All-Pair Shortest Path Problem on Interval Graphs”, *Networks*, Vol. 22, pp. 21-35, 1992.
 - [16] K. Takamizawa, T. Nishizeki, and N. Saito, “Linear-Time Computability of Combinatorial Problems on Series-Parallel Graphs”, *Journal of the A.C.M.*, Vol. 29, No. 3, pp. 623-641, 1982.
 - [17] T. Takaoka, “An $O(n^3 \log \log n / \log n)$ Time Algorithm for the All-Pairs Shortest Path Problem”, *Information Processing Letters.*, Vol. 96, pp. 155-161, 2005.
 - [18] M. Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time”, *Journal of the ACM*, Vol. 11, No. 1, 1999.
 - [19] C-KYen and C-Y Tang, *A Study on the Searchlight Guarding Problem*, Ph.D. Dissertation, Department of Computer Science, National Tsing Hua University, 1996.